

Por que não usar PostgreSQL na sua stack de desenvolvimento?

PGDay Blumenau 2026

Dickson Guedes

May 30, 2026

Por que não usar PostgreSQL na sua stack de desenvolvimento?

- PGDay Blumenau 2026
- Dickson S. Guedes
 - `guedes@timbira.com.br`
 - `https://github.com/guedes`

Sobre esta palestra

- Antes de adicionar mais uma ferramenta à stack, vale perguntar se o PostgreSQL já resolve o problema com simplicidade suficiente.
- A ideia não é defender que tudo deve ser feito no PostgreSQL, mas trazer critérios para avaliar quando ele já entrega uma solução coerente, simples e operacionalmente viável.
- O que trago aqui é uma lista não exaustiva de funcionalidades importantes e onde elas podem ser aplicadas.
- A pergunta não é: "dá para fazer no PostgreSQL?" mas "se o PostgreSQL resolve isso com simplicidade suficiente antes de eu assumir o custo de mais uma ferramenta?"

Observação sobre PostgreSQL 19

Os tópicos marcados como PostgreSQL 19 devem ser tratados como recursos de versão em desenvolvimento até a publicação oficial da versão estável.

1. JSON / JSONB para dados semiestruturados

Por que usar

Com JSON / JSONB...

- você armazena dados flexíveis sem abandonar o modelo relacional;
- fica mais fácil lidar com partes do domínio que têm estrutura variável, como eventos, configurações ou payloads de integração.

Onde usar

- Eventos de aplicação
- Configurações por cliente
- Payloads recebidos de APIs externas
- Atributos opcionais de entidades

Exemplo

```
CREATE TABLE eventos (  
  id bigserial PRIMARY KEY,  
  payload jsonb NOT NULL  
);  
  
SELECT *  
FROM eventos  
WHERE payload @> '{"tipo": "checkout"}';
```

Ponto de atenção

Se tudo vira JSONB...

- o modelo perde clareza;
- você perde constraints e tipagem explícita;
- a governança do modelo tende a piorar.

2. Full Text Search (Busca textual nativa)

Por que usar

Evita adicionar Elasticsearch ou OpenSearch cedo demais para buscas internas simples ou médias.

Onde usar

- Busca em artigos
- FAQ
- Catálogo interno
- Documentação
- Produtos ou posts

Exemplo

```
SELECT *  
FROM posts  
WHERE to_tsvector('portuguese', titulo || ' ' || corpo)  
      @@ plainto_tsquery('portuguese', 'backup postgres');
```

Ponto de atenção

Quando a busca começa a ficar mais sofisticada...

- relevância complexa pode justificar outro motor;
- autocomplete sofisticado pode justificar outro motor;
- análises de busca mais avançadas podem justificar outro motor.

3. Índices GIN para busca, JSONB e arrays

Por que usar

Acelera consultas em estruturas compostas, como documentos JSONB, arrays e vetores de busca textual.

Onde usar

- Full Text Search
- Filtros em JSONB
- Busca por tags
- Arrays consultados por contenção

Exemplo

```
CREATE INDEX idx_posts_busca  
ON posts  
USING GIN (to_tsvector('portuguese', titulo || ' ' || corpo));
```

Ponto de atenção

Ao adotar índices GIN...

- o custo de escrita pode aumentar;
- o custo de manutenção pode aumentar;
- o consumo de espaço em disco pode crescer.

4. Arrays para listas simples

Por que usar

Permite representar listas pequenas e simples sem criar tabelas auxiliares.

Onde usar

- Tags simples
- Flags
- Pequenas listas de categorias
- Valores auxiliares de configuração

Exemplo

```
CREATE TABLE artigos (  
    id bigserial PRIMARY KEY,  
    titulo text,  
    tags text[]  
);  
  
SELECT *  
FROM artigos  
WHERE tags @> ARRAY['postgres', 'databases'];
```

Ponto de atenção

Quando o relacionamento deixa de ser simples...

- relacionamentos importantes geralmente merecem modelagem normalizada;
- relacionamentos auditáveis geralmente merecem modelagem normalizada;
- relacionamentos muito consultados geralmente merecem modelagem normalizada.

5. Constraints como regras de integridade

Por que usar

Com constraints...

- você mantém regras críticas próximas dos dados;
- diminui a chance de inconsistência causada por múltiplas aplicações acessando o mesmo banco.

Onde usar

- Estados válidos
- Valores mínimos e máximos
- Unicidade
- Relacionamentos obrigatórios
- Regras de integridade de domínio

Exemplo

```
CREATE TABLE pedidos (  
  id bigserial PRIMARY KEY,  
  total numeric CHECK (total >= 0),  
  status text CHECK (status IN ('novo', 'pago', 'cancelado'))  
);
```

Ponto de atenção

Constraints funcionam melhor quando...

- representam invariantes claras;
- a regra não muda o tempo todo na aplicação.

6. Views para encapsular consultas

Por que usar

Com views...

- você padroniza consultas usadas por relatórios, APIs internas e módulos da aplicação;
- evita duplicar SQL em vários lugares.

Onde usar

- Relatórios
- APIs de leitura
- Camadas de compatibilidade
- Modelos simplificados para BI

Exemplo

```
CREATE VIEW vw_pedidos_abertos AS  
SELECT id, cliente_id, total, criado_em  
FROM pedidos  
WHERE status = 'aberto';
```

Ponto de atenção

Quando as views começam a se empilhar...

- o custo pode ficar escondido;
- o troubleshooting pode ficar mais difícil;
- as dependências podem ficar frágeis.

7. Materialized views como cache de leitura

Por que usar

Com materialized views...

- você mantém resultados pré-calculados dentro do banco;
- adia a necessidade de adicionar outro componente de cache ou busca.

Onde usar

- Dashboards
- Agregados por período
- Relatórios pesados
- Consultas recorrentes de leitura

Exemplo

```
CREATE MATERIALIZED VIEW mv_vendas_por_dia AS  
SELECT date(created_at) AS dia, sum(total) AS total  
FROM pedidos  
GROUP BY 1;
```

```
REFRESH MATERIALIZED VIEW mv_vendas_por_dia;
```

Ponto de atenção

Antes de tratar materialized view como cache...

- é preciso definir a estratégia de atualização;
- é preciso definir como lidar com concorrência;
- é preciso definir a tolerância a dados defasados.

8. Triggers para automações próximas do dado

Por que usar

Com triggers...

- você centraliza comportamentos transversais (aplicações diferentes);
- garante que eles aconteçam independentemente da aplicação que alterou os dados.

Onde usar

- Auditoria
- Atualização de timestamps
- Histórico de mudanças
- Validações transacionais

Exemplo

```
CREATE FUNCTION set_updated_at()  
RETURNS trigger AS $$  
BEGIN  
    NEW.updated_at = now();  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_updated_at  
BEFORE UPDATE ON clientes  
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Ponto de atenção

Quando o comportamento fica escondido em triggers...

- para quem desenvolve, esse comportamento pode não ser óbvio;
- a manutenção pode ficar mais difícil.

9. Functions e procedures para operações transacionais

Por que usar

Com functions e procedures...

- você aproxima a lógica transacional dos dados;
- reduz múltiplas idas da aplicação ao banco.

Onde usar

- Rotinas transacionais
- Processamento interno
- Cálculos reutilizados
- Operações críticas com atomicidade

Exemplo

```
CREATE FUNCTION cancelar_pedido(p_id bigint)
RETURNS void AS $$
BEGIN
    UPDATE pedidos
    SET status = 'cancelado'
    WHERE id = p_id AND status = 'aberto';
END;
$$ LANGUAGE plpgsql;
```

Ponto de atenção

Quando a lógica começa a se concentrar demais no banco...

- um monólito escondido pode surgir;
- o versionamento pode ficar mais difícil.

10. LISTEN / NOTIFY para eventos leves

Por que usar

Permite sinalização assíncrona simples entre sessões conectadas ao banco.

Onde usar

- Invalidar cache
- Avisar workers
- Sinalizar novos eventos
- Atualizações em tempo quase real

Exemplo

```
NOTIFY novo_pedido, '{"pedido_id": 123}';
```

```
LISTEN novo_pedido;
```

Ponto de atenção

Se o fluxo é crítico...

- não é uma fila durável;
- não substitui um broker.

11. SKIP LOCKED para filas simples

Por que usar

Permite que múltiplos workers consumam tarefas sem processar a mesma linha ao mesmo tempo.

Onde usar

- Jobs internos
- Filas simples
- Processamento assíncrono leve
- Retentativas controladas pela aplicação

Exemplo

```
SELECT id
  FROM jobs
 WHERE status = 'pendente'
 ORDER BY id
  FOR UPDATE SKIP LOCKED
 LIMIT 1;
```

Ponto de atenção

Quando a fila deixa de ser simples...

- quando o mesmo evento precisa ser consumido por vários processos (fanout), um broker pode fazer mais sentido;
- ordering forte pode exigir um broker;
- grande volume pode exigir um broker;
- políticas complexas de retry podem exigir um broker.

12. Advisory locks para coordenação simples

Por que usar

Com advisory locks...

- você coordena processos usando o próprio banco;
- evita adicionar Redis, ZooKeeper ou etcd para casos pequenos.

Onde usar

- Job singleton
- Agendador único
- Proteção contra execução concorrente
- Locks por recurso lógico

Exemplo

```
SELECT pg_try_advisory_lock(42);
```

Ponto de atenção

Ao usar advisory locks...

- é preciso disciplina para liberar os locks;
- é preciso entender o comportamento por sessão ou transação.

13. Row Level Security para isolamento por linha

Por que usar

Reforça segurança diretamente na camada de dados, especialmente em aplicações multi-tenant.

Onde usar

- SaaS multi-tenant
- Dados por organização
- Dados por conta
- Ambientes com defesa em profundidade

Exemplo

```
ALTER TABLE documentos ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY docs_por_cliente  
ON documentos  
USING (cliente_id = current_setting('app.cliente_id')::int);
```

Ponto de atenção

Se a RLS for mal configurada...

- ela pode criar uma falsa sensação de segurança;
- testes específicos passam a ser indispensáveis.

14. Schemas para organização lógica

Por que usar

Com schemas...

- você separa módulos, domínios e responsabilidades;
- evita criar vários bancos ou serviços.

Onde usar

- Separação por domínio
- Auditoria
- Módulos internos
- Staging e integração

Exemplo

```
CREATE SCHEMA financeiro;  
CREATE SCHEMA auditoria;
```

```
CREATE TABLE financeiro.faturas (...);  
CREATE TABLE auditoria.eventos (...);
```

Ponto de atenção

Ao organizar o banco com schemas...

- isso não vira isolamento físico;
- isso não substitui uma política de permissões bem desenhada.

15. Extensões como mecanismo de expansão

Por que usar

Com extensões...

- você adiciona capacidades ao PostgreSQL;
- pode evitar operar outro serviço.

Onde usar

- Busca aproximada
- Integração
- Tipos especializados
- Funções adicionais
- Observabilidade
- Agendamentos

Exemplo

```
CREATE EXTENSION pg_trgm;  
CREATE EXTENSION unaccent;  
CREATE EXTENSION pg_cron;  
CREATE EXTENSION postgres_fdw;
```

Ponto de atenção

Ao instalar uma extensão...

- você adiciona mais uma dependência;
- suporte e upgrade precisam ser avaliados;
- segurança e disponibilidade no ambiente gerenciado precisam ser avaliadas.

16. pg_trgm para busca aproximada

Por que usar

Resolve buscas tolerantes a erro e autocomplete simples sem adicionar motor de busca.

Onde usar

- Busca por nome
- Busca por produto
- Autocomplete simples
- Deduplicação aproximada

Exemplo

```
SELECT nome  
FROM clientes  
WHERE nome % 'Guedes';
```

Ponto de atenção

Quando a experiência de busca precisa evoluir...

- ranking sofisticado pode exigir outra solução;
- relevância complexa pode exigir outra solução;
- uma experiência de busca mais elaborada pode exigir outra solução.

17. postgres_{fdw} para consulta federada

Por que usar

Com postgres_{fdw}...

- você consulta outro PostgreSQL via SQL;
- viabiliza integração e migração gradual.

Onde usar

- Integração entre sistemas
- Migração gradual
- Consultas federadas internas
- Consolidação temporária de dados

Exemplo

```
CREATE EXTENSION postgres_fdw;
```

```
-- Depois: criar server, user mapping e foreign table.
```

```
SELECT *
```

```
FROM tabela_remota;
```

Ponto de atenção

Ao consultar outro PostgreSQL via FDW...

- a latência precisa ser bem avaliada;
- o acoplamento precisa ser bem avaliado;
- transações distribuídas e permissões precisam ser bem avaliadas.

18. Logical replication para integração e migração

Por que usar

Com logical replication...

- você replica subconjuntos de dados por tabela;
- facilita migrações, integrações e arquiteturas de leitura.

Onde usar

- Migração sem parada longa
- CDC básico
- Integração entre sistemas
- Cópias seletivas de dados

Exemplo

```
CREATE PUBLICATION pub_app  
FOR TABLE clientes, pedidos;
```

Ponto de atenção

Ao entrar em replicação lógica...

- o desenho operacional continua existindo;
- slots e lag precisam ser monitorados;
- DDL, conflitos e retenção de WAL precisam ser monitorados.

19. Generated columns para dados derivados

Por que usar

Mantém cálculos derivados consistentes e próximos dos dados.

Onde usar

- Campos calculados
- Normalização de busca
- Chaves derivadas
- Expressões reutilizadas

Exemplo

```
CREATE TABLE pessoas (  
    nome text,  
    sobrenome text,  
    nome_completo text GENERATED ALWAYS AS (nome || ' ' || sobrenome) STORED  
);
```

```
CREATE TABLE pessoas_virtual (  
    nome text,  
    sobrenome text,  
    nome_completo text GENERATED ALWAYS AS (nome || ' ' || sobrenome) VIRTUAL  
);
```

Ponto de atenção

Quando a coluna é derivada...

- nem toda regra deve ir para o banco;
- a derivação precisa ser estável e clara.
- ‘STORED’ grava o valor e calcula na escrita; ‘VIRTUAL’ calcula na leitura;
- em PostgreSQL 18+, se você omitir ‘STORED’ / ‘VIRTUAL’, o padrão é ‘VIRTUAL’;
- ‘VIRTUAL’ tem mais restrições e funciona melhor em expressões simples com recursos nativos.

20. PostgreSQL 19: SQL/PGQ para consultas de grafo

Por que usar

Permite expressar alguns problemas de relacionamento como padrões de grafo dentro do PostgreSQL.

Onde usar

- Redes de relacionamento
- Dependências entre objetos
- Relações entre contas
- Hierarquias
- Recomendações simples

Exemplo

```
-- Exemplo conceitual:  
-- usuarios(id, nome) e segue(perfil_origem_id, perfil_destino_id)
```

```
CREATE PROPERTY GRAPH social_graph  
  VERTEX TABLES (  
    usuarios KEY (id)  
  )  
  EDGE TABLES (  
    segue  
      KEY (perfil_origem_id, perfil_destino_id)  
      SOURCE KEY (perfil_origem_id) REFERENCES usuarios (id)  
      DESTINATION KEY (perfil_destino_id) REFERENCES usuarios (id)  
  );
```

```
SELECT *  
FROM GRAPH_TABLE (  
  social_graph  
  MATCH (u IS usuarios)-[s IS segue]->(v IS usuarios)  
  COLUMNS (u.nome AS perfil_origem, v.nome AS perfil_destino)  
);
```

Ponto de atenção

- um banco de grafos especializado ainda pode fazer mais sentido quando o problema gira em torno de travessias complexas.

21. PostgreSQL 19: UPDATE / DELETE FOR PORTION para operações de range temporais

Por que usar

Com 'UPDATE / DELETE FOR PORTION'...

- você simplifica alterações em dados com período de validade;
- reduz lógica manual na aplicação.

Onde usar

- Contratos
- Preços com vigência
- Assinaturas
- Políticas temporais
- Histórico de validade

Exemplo

```
UPDATE contratos
FOR PORTION OF periodo FROM DATE '2026-01-01' TO DATE '2026-06-30'
SET valor = 1200
WHERE cliente_id = 10;
```

Ponto de atenção

Quando o tempo vira parte central do domínio...

- isso só vale quando a modelagem temporal é clara;
- times sem domínio desse tema podem se confundir.

22. PostgreSQL 19: GROUP BY ALL para agregações menos repetitivas

Por que usar

Com 'GROUP BY ALL'...

- você melhora a ergonomia de consultas analíticas simples;
- agrupa automaticamente colunas não agregadas da lista de seleção.

Onde usar

- Relatórios rápidos
- Consultas exploratórias
- Dashboards internos
- Agregações simples

Exemplo

```
SELECT cliente_id, status, count(*)  
FROM pedidos  
GROUP BY ALL;
```

Ponto de atenção

- a intenção da query pode ficar escondida se o recurso for usado sem critério.

23. PostgreSQL 19: IGNORE NULLS / RESPECT NULLS em window functions

Por que usar

Com 'IGNORE NULLS / RESPECT NULLS'...

- você facilita análise de séries e eventos;
- evita contornos complexos em SQL.

Onde usar

- Séries temporais
- Métricas
- Análise de eventos
- Preenchimento lógico de valores

Exemplo

```
SELECT
  dia,
  valor,
  last_value(valor) IGNORE NULLS OVER (ORDER BY dia)
FROM metricas;
```

Ponto de atenção

Ao usar esse recurso analítico...

- ele continua sendo só um recurso analítico;
- não substitui governança de dados;
- não substitui qualidade na ingestão.

24. PostgreSQL 19: INSERT ON CONFLICT DO SELECT para upsert com retorno da linha conflitante

Por que usar

Com 'INSERT ON CONFLICT DO SELECT'...

- a aplicação pode recuperar a linha existente em caso de conflito;
- evita um fluxo extra.

Onde usar

- Cadastro idempotente
- APIs
- Importações
- Sincronização de dados

Exemplo

```
INSERT INTO usuarios (email, nome)
    VALUES ('a@x.com', 'Ana')
ON CONFLICT (email)
DO SELECT
    RETURNING id, email, nome;
```

Ponto de atenção

Mesmo com a ergonomia melhor...

- concorrência ainda exige cuidado;
- locks ainda exigem cuidado;
- a semântica do conflito ainda exige cuidado.

25. PostgreSQL 19: MERGE / SPLIT de partições

Por que usar

Com 'MERGE / SPLIT'...

- você facilita a evolução de estratégias de particionamento;
- torna a manutenção dessas estratégias mais simples.

Onde usar

- Tabelas particionadas por data
- Retenção de dados
- Reorganização de partições
- Ambientes com alto volume

Exemplo

```
ALTER TABLE eventos  
MERGE PARTITIONS (...);
```

```
ALTER TABLE eventos  
SPLIT PARTITION (...);
```

Ponto de atenção

Mesmo com operações mais fáceis...

- particionamento continua sendo uma decisão arquitetural;
- mal usado, pode piorar a operação;
- mal usado, pode piorar o planejamento de queries.

26. PostgreSQL 19: COPY TO em formato JSON

Por que usar

Facilita exportar resultados SQL para integrações, scripts e APIs internas.

Onde usar

- Exportações
- Pipelines simples
- Debug
- Integrações internas
- Geração de arquivos JSON

Exemplo

```
COPY (  
  SELECT id, nome, email  
  FROM clientes  
) TO STDOUT WITH (FORMAT JSON);
```

Ponto de atenção

- não substitui um contrato de API quando o consumo é público ou crítico.

27. PostgreSQL 19: COPY FROM com ON_ERROR SET_NULL para carga tolerante a valores inválidos

Por que usar

Permite transformar valores inválidos em NULL durante ingestões simples.

Onde usar

- Cargas em staging
- CSVs semi-sujos
- Protótipos de ingestão
- Importações operacionais

Exemplo

```
COPY staging_clientes
FROM '/dados/clientes.csv'
WITH (
  FORMAT csv,
  HEADER true,
  ON_ERROR SET_NULL
);
```

Ponto de atenção

- problemas de qualidade de dados podem ficar escondidos se a carga tolerar erro sem auditoria.

28. PostgreSQL 19: Logical Replication com replicação lógica de sequences

Por que usar

Reduz uma dor operacional comum: tabelas replicadas sem alinhamento adequado de sequences.

Onde usar

- Migrações
- Integrações
- Ambientes com replicação lógica
- Cópias seletivas com escrita futura

Exemplo

```
CREATE PUBLICATION app_pub  
FOR ALL TABLES, ALL SEQUENCES;
```

Ponto de atenção

Mesmo com replicação lógica de sequences...

- o cutover continua precisando de planejamento;
- conflitos continuam precisando de planejamento;
- escrita concorrente continua precisando de planejamento.

Categorizando

Modelagem e regras perto do dado

- 1. JSONB
- 4. Arrays
- 5. Constraints
- 13. Row Level Security
- 14. Schemas
- 19. Generated Columns
- 20. SQL/PGQ
- 21. UPDATE / DELETE FOR PORTION

Leitura, consulta e análise sem sair do PostgreSQL

- 2. Full Text Search
- 3. Índices GIN
- 6. Views
- 7. Materialized Views
- 16. pgtrgm
- 22. GROUP BY ALL
- 23. IGNORE NULLS / RESPECT NULLS

Automação e coordenação sem adicionar mais infraestrutura

- 8. Triggers
- 9. Functions / Procedures
- 10. LISTEN / NOTIFY
- 11. SKIP LOCKED
- 12. Advisory Locks
- 24. INSERT ON CONFLICT DO SELECT
- 27. COPY FROM com ON_{ERROR} SET_{NULL}

Integração, distribuição e evolução da arquitetura

- 15. Extensões
- 17. `postgres_fdw`
- 18. Logical Replication
- 25. MERGE / SPLIT de partições
- 26. COPY TO em formato JSON
- 28. Replicação lógica de sequences

Fim

- guedes@timbira.com.br
- <https://gitHub.com/guedes>

